Memory Reclaim Prediction Project Explanation

What Problem Are We Solving

When you play a heavy mobile game like BGMI or CODM for 20-30 minutes, the game competes with everything else running in the background — music apps, messaging apps, browser tabs, system services. The operating system has to keep all of these alive while giving the game maximum memory to run smoothly.

At some point the system runs out of breathing room. The memory manager — a kernel component called LMKD (Low Memory Killer Daemon) — steps in and reclaims memory by killing or trimming background processes. This reclaim event is what keeps the game alive and running.

The problem is nobody knows in advance how much memory will be reclaimed. Sometimes it reclaims 50MB. Sometimes 800MB. This unpredictability creates two real problems:

Too little reclaim:
- Game does not get enough free memory
- Textures start dropping from GPU memory
- Frame rate falls below acceptable threshold
- User experiences stutter, freezes, and lag

Too much reclaim:
- System kills more background apps than necessary
- User switches back to WhatsApp after gaming
- App has to reload completely from scratch
- User opens browser — all tabs are gone
- Extra battery consumed reloading killed processes

If you could predict how much memory will be reclaimed before it happens, the system could pre-free exactly the right amount proactively — keeping the game smooth without unnecessarily killing background apps the user actually cares about.

—----------

Why This Problem Matters

Gaming is one of the top three use cases on modern Android devices alongside camera and productivity. Memory management during gaming directly affects three things users care about deeply — frame rate stability during gameplay, the experience of switching between apps mid-game, and how quickly battery drains. Getting reclaim prediction right means better game performance with less collateral damage to other running applications.

Every Android device manufacturer deals with this problem. The kernel's LMKD uses fixed thresholds and simple heuristics to decide when and how much to reclaim. It has no understanding of which game is running, what graphics quality is selected, how long the session has been active, or how hot the processor is. It reacts blindly to memory levels crossing predefined limits.

An ML model that understands all of these signals together can predict reclaim magnitude far more accurately than any fixed threshold approach.

—----------

How We Collected the Data

We used ADB (Android Debug Bridge) — a standard Android developer tool — to pull device telemetry directly from real mobile hardware during active gaming sessions. ADB reads kernel-level files that the operating system writes in real time. No special root access is needed. No modification to the device is required.

We collected across four device series covering three RAM configurations — 6GB, 8GB, and 12GB devices. We played 12 different games ranging from ultra-high-memory AAA titles like BGMI, CODM, and eFootball down to casual low-memory games like Candy Crush, Angry Bird, and Hill Climb Racing. Each session ran for 10 to 45 minutes of real gameplay.

The files we read from the device:

/proc/meminfo → total memory, swap state, cached memory
DMA buffer stats → GPU shared memory usage
Java heap metrics → runtime memory allocation state
/sys/class/thermal/ → application processor temperature
/proc/stat → CPU utilization
App package info → APK size, compilation state, odex status

Beyond standard memory counters, we captured game context information that most memory studies never include — graphics settings selected by the user, how the app was compiled at install time, whether bytecode is pre-optimized, screen resolution, refresh rate, and whether the session was online or offline. This context makes the dataset uniquely rich.

Total dataset: 60,000 rows across 1,086 sessions.
Each row is one memory snapshot during an active gaming session.

The Research Question

"Given the current device memory state, game context,
and hardware configuration at a snapshot moment during
active gaming, can we accurately predict how many KB
of memory will be reclaimed by the kernel — and which
signals drive reclaim magnitude most strongly across
different game types and device RAM configurations?"

This is interesting for three reasons. First, reclaim magnitude depends on memory state, thermal state, game type, graphics quality, and device RAM all simultaneously — no single signal explains it. Second, the relationship between these signals and reclaim magnitude is highly non-linear. High graphics on a 6GB device under thermal stress behaves completely differently from high graphics on a 12GB device at ambient temperature. Third, the finding of which signal matters most has direct engineering implications for how LMKD should be tuned per game category.

—----------

What Each Column in the Dataset Means

This is where the project is genuinely unique. Most memory studies only capture raw memory counters. We captured the complete picture.

Game and Device Context

game:
Which title is being played. We covered 12 games spanning a wide memory footprint range — BGMI and CODM at the high end consuming 2.5GB+, Candy Crush and Angry Bird at the low end consuming under 300MB. This diversity is intentional — the model must learn patterns that generalize across game types, not just memorize one game's behavior.

game_category:
Action, sports, casual, or strategy. Games within the same category share memory access patterns. Action games have rapid texture streaming and frequent GPU buffer swaps. Casual games have static memory footprints with occasional GC events. Strategy games have large persistent state with moderate streaming. Encoding category gives the model a higher-level signal beyond just game name.

graphics_setting:
Low, medium, high, or ultra. This turned out to be the second most important feature in the entire project. Higher graphics settings push more data into GPU memory, increase DMA buffer usage, and generate more memory pressure per frame. A game at ultra settings on a 6GB device creates fundamentally different memory conditions than the same game at low settings. No existing memory reclaim study includes this signal.

compilation_filter:
How the app was compiled at install time — speed, everything, verify, quicken, or speed-profile. This affects how efficiently the app uses memory at runtime. An app compiled with "everything" has more pre-compiled native code and lower memory overhead at runtime. An app compiled with "verify" runs more interpreted code and uses more heap memory. This signal captures the app's runtime memory efficiency class.

odex_vdex:
Whether the app's bytecode is pre-compiled into native format. odex+vdex means fully pre-compiled — fastest execution, lowest memory overhead. none means fully interpreted — highest memory overhead. This directly affects how much heap memory the app consumes during gameplay.

resolution and refreshrate:
Higher resolution means more framebuffer memory. Higher refresh rate means more frequent GPU buffer swaps. A game running at 1440×3088 at 120Hz uses significantly more GPU memory than the same game at 720×1560 at 60Hz. These signals capture the rendering workload intensity.

multiuser and online_offline:
Multiplayer online games maintain network buffers, game state synchronization structures, and voice chat buffers that offline single-player games do not. Online multiplayer sessions have consistently higher memory overhead than offline sessions of the same game.

apk_size_mb:
Larger games have more assets to stream into memory during gameplay. A 2100MB game like eFootball has far more texture variety to load than a 85MB game like Candy Crush. APK size is a proxy for asset complexity and streaming pressure.

duration_sec:
How long the session ran. Longer sessions accumulate more memory fragmentation, more GC cycles, and more background process accumulation. Memory pressure typically increases over the course of a long session.

Raw Memory Signals

memtotal_kb:
Total physical RAM on the device. 6GB = 6,291,456 KB. 8GB = 8,388,608 KB. 12GB = 12,582,912 KB. This is a device constant that normalizes other features.

swaptotal_kb and swapfree_kb:
Swap is disk space used as overflow memory when physical RAM is full. When swapfree_kb starts dropping significantly the system is under extreme memory pressure — physical RAM is exhausted and the kernel is using slower flash storage. High swap usage almost always precedes large reclaim events.

dma_buf_kb:
Direct Memory Access buffer — memory shared between the CPU and GPU. When a game renders a frame the GPU reads textures from DMA buffers. High DMA buffer usage means the GPU is actively consuming significant memory that is not available to other processes. Games with high graphics settings have proportionally higher DMA buffer usage.

heapsize_kb, heapmaxfree_kb, heapminfree_kb:
Java heap metrics that capture runtime memory state. heapsize_kb is the total heap allocated to the app. heapmaxfree_kb is the largest contiguous free chunk in the heap — a small value means the heap is fragmented even if total free space exists. heapminfree_kb is the threshold below which the garbage collector fires. Together these three metrics tell you exactly how stressed the Java runtime is.

dha_empty_kb and dha_cache_kb:
Device Heap Allocator metrics. dha_cache_kb is cached memory that can be reclaimed quickly if needed. High dha_cache_kb relative to dha_total_kb means the system has a meaningful pool of reclaimable cached memory available — reclaim events will be more effective.

current_cpuload:
CPU utilization at the snapshot moment. High CPU load combined with high memory pressure creates the worst conditions for smooth gameplay — both compute and memory resources are saturated simultaneously.

aptemp:
Application processor temperature in Celsius. As the device heats up the thermal management system starts making decisions that interact with memory management. A hot device under memory pressure behaves differently from a cool device under the same memory pressure. This interaction ended up as the third most important SHAP feature.

Target

reclaimed_kb:
How much memory in KB was reclaimed by the kernel at this snapshot. This is what we predict. Values ranged from 30,000 KB (29 MB) to 936,803 KB (915 MB) in the dataset with a mean of around 346 MB.

—----------

Why Single-Stage Regression and Not Two Models

Some ML projects on similar problems use a two-stage approach — first classify whether a significant reclaim will happen, then regress on the magnitude. We deliberately chose a single-stage regression for this project.

The reason is straightforward. During active gaming reclaim always happens — it is not a question of whether but of how much. Every gaming session long enough to generate a snapshot will have reclaim events. The interesting and actionable question is purely about magnitude. How much memory will be freed? That is a continuous value prediction — a regression problem from the start.

Adding a classifier first would introduce unnecessary complexity without adding information. The classifier would essentially always predict yes during gaming sessions, making it useless as a gate. Single-stage regression is the architecturally correct choice for this problem framing.

—----------

Why XGBoost

We had multiple options — Linear Regression, Random Forest, LightGBM, XGBoost, or neural networks. Here is the reasoning for choosing XGBoost.

Linear Regression fails here because the relationship between memory signals and reclaim magnitude is fundamentally non-linear. A device at 85% memory utilization does not reclaim proportionally more than a device at 70% — it reclaims exponentially more as pressure approaches physical limits. No linear model captures this.

Random Forest would work but XGBoost is consistently stronger on regression tasks with this kind of tabular sensor data. XGBoost's gradient boosting builds each tree to correct the errors of all previous trees, making it particularly good at learning complex interactions between features like the interplay between graphics settings, device RAM, and memory utilization.

LightGBM is faster but XGBoost's default regularization parameters (reg_lambda=1.0) are better tuned for regression tasks where preventing overfitting on the continuous target is more nuanced. For a dataset of 60,000 rows with 34 features, training speed is not a bottleneck so we optimize for regression accuracy over training speed.

Neural networks were not considered because they require significantly more data and hyperparameter tuning to outperform tree-based models on tabular sensor data. The interpretability of XGBoost plus SHAP is also a significant advantage over black-box neural networks when the goal includes explainability and research findings.

—----------

Why No Rolling Window Features

Thermal prediction used rolling features heavily — 30-second and 60-second temperature trends. Memory reclaim prediction does not need them. Here is why.

In thermal prediction the pattern of change over time is the signal. Temperature rising at 1.5°C per second is more dangerous than temperature sitting at 70°C. The trajectory matters more than the current value.

In memory reclaim prediction the current snapshot state is the signal. The kernel's LMKD makes reclaim decisions based on current memory levels crossing thresholds. The amount it reclaims depends on how much memory is needed right now, not on how memory changed over the last 30 seconds. The current utilization, swap state, heap condition, and game context fully characterize the reclaim magnitude without needing temporal history.

This is also why each row in the dataset is relatively independent. The reclaim amount at 10:00:00 is not strongly dependent on the reclaim amount at 09:59:30. Contrast this with temperature which is physically continuous — you cannot jump from 60°C to 75°C in one 5-second step.

Train/Test Split — Why Session-Based

We split by sessions, not by rows. All rows from a session go entirely into training or entirely into tests — never split between them.

Why? Within a session rows are from the same device running the same game in the same conditions. If you put row 50 of a session in training and row 48 in test, the model trains on data from after the test point. That is temporal leakage and it makes validation accuracy artificially high. In real deployment you would never have future data when making a prediction.

By splitting entire sessions you simulate real deployment — the model has never seen any data from the test device sessions. First 80% of sessions (868 sessions, 47,634 rows) go to training. Last 20% (218 sessions, 12,366 rows) go to test. Zero session overlap confirmed programmatically.

Scaling — Why and How

XGBoost is a tree-based model and technically does not need feature scaling for prediction accuracy. Trees split on feature values and the scale of those values does not affect where optimal splits are found.

We scale anyway for one important reason — SHAP value comparability. When features are on completely different scales (memtotal_kb in millions vs dha_cache_ratio between 0 and 1), SHAP values are hard to compare directly. Scaling brings all features to zero mean and unit variance so SHAP magnitudes are directly comparable across features, making the feature importance analysis honest and interpretable.

The scaler is fitted on training data only. The same fitted scaler is applied to test data. Test data statistics never influence the scaling parameters.

—----------

Cross Validation — Why TimeSeriesSplit

Standard k-fold cross-validation randomly shuffles data before splitting into folds. For session data this is wrong. If fold 3 training contains rows from session_042 and fold 3 validation also contains rows from session_042, the model validates on data from the same session it trained on. That inflates validation scores.

TimeSeriesSplit keeps validation data chronologically after training data within each fold. Fold 1 trains on the first 20% of rows and validates on the next 20%. Fold 2 trains on the first 40% and validate on the next 20%. Each validation window is always temporally after the training window. This is the correct approach for any data that has a natural time ordering.

Results across 5 folds:

Fold 1: MAE = 42,255 KB
Fold 2: MAE = 36,679 KB
Fold 3: MAE = 35,005 KB
Fold 4: MAE = 34,100 KB
Fold 5: MAE = 38,547 KB
Mean : 37,317 KB ± 2,898 KB

Variance across folds is relatively low (std = 2,898 KB vs mean = 37,317 KB) which indicates the model is stable and not highly sensitive to which sessions end up in validation.

—----------

Model Results — What the Numbers Mean

Global Metrics

MAE   : 42,429 KB (41.4 MB)
RMSE   : 52,563 KB (51.3 MB)
Median : 36,493 KB (35.6 MB)

MAE of 41MB means on average the model's prediction is off by 41MB from the actual reclaim amount. Given that reclaim ranges from 29MB to 915MB across the dataset, a 41MB average error represents about 4.5% of the full range. That is strong accuracy.

The gap between MAE (41MB) and Median Error (35MB) is small, which means the error distribution is relatively symmetric without extreme outliers pulling the mean up. This is healthier than a situation where median is very low and mean is very high (which would indicate a few catastrophically wrong predictions).

Within tolerance breakdown:

Within ±50MB : 66.3%
Within ±100MB : 95.2%
Within ±150MB : 99.8%
Within ±200MB : 100.0%

The most operationally important number is 95.2% within ±100MB. For practical use by a memory management system, being within 100MB is sufficient to make the right pre-freeing decision. If the system needs to free approximately 400MB and our model predicts 380MB or 420MB, it makes the same pre-freeing decision either way. 95.2% accuracy at this tolerance level means the system would make the right decision in 19 out of every 20 reclaim events.

Baseline Comparison

Naive mean baseline MAE : 134,075 KB (131 MB)
XGBoost model MAE : 42,429 KB (41 MB)
Improvement : 68.4%

The naive baseline simply predicts the training set mean (around 346 MB) for every single row regardless of game, device, or memory state. Our model outperforms this by 68.4%. This is a strong result and the most important number for the resume because it proves ML adds massive real value over the simplest possible approach.

—----------

Per-Game Analysis — The Research Story

Easiest to predict:
HillClimbRacing MAE = 38,913 KB (38 MB)
CandyCrush MAE = 39,245 KB (38 MB)
LudoKing MAE = 39,383 KB (39 MB)

Hardest to predict:
CODM MAE = 49,018 KB (48 MB)
FreeFire MAE = 47,157 KB (46 MB)
eFootball MAE = 44,091 KB (43 MB)

This pattern is not random. Low-memory casual games (HillClimbRacing, CandyCrush) have stable and predictable memory footprints. Their reclaim events are driven by simple memory level thresholds and follow consistent patterns. High-memory action games (CODM, FreeFire) have dynamic memory behavior — rapid texture streaming, variable network buffer sizes, frequent GPU buffer swaps — that creates more volatile reclaim patterns that are harder to predict precisely.

This is a research finding:
> "Reclaim magnitude is more predictable for low-memory casual games than for high-memory action titles, suggesting that memory volatility from dynamic asset streaming in AAA mobile games introduces prediction uncertainty that static memory footprint games do not exhibit."

—----------

Per-Graphics Analysis — Another Research Finding

Low graphics MAE = 38,592 KB (38 MB) ← best
Medium graphics MAE = 39,987 KB (39 MB)
High graphics MAE = 43,631 KB (43 MB)
Ultra graphics MAE = 47,286 KB (46 MB) ← worst

Performance degrades consistently as graphics quality increases. Ultra settings push devices into non-linear memory behavior — DMA buffers expand and contract rapidly, GPU memory pressure creates backpressure on system memory, and the interaction between thermal state and memory management becomes more complex. This non-linearity is harder for the model to capture precisely.

Research finding:
> "Prediction error increases monotonically with graphics quality setting, suggesting that ultra graphics settings introduce non-linear memory pressure dynamics that are less predictable than the near-linear behavior observed at lower graphics settings."

—----------

SHAP Explainability — Why Each Prediction Was Made

SHAP (SHapley Additive exPlanations) tells you how much each feature contributed to each individual prediction. For every prediction it gives every feature a positive or negative contribution score. Features that pushed the prediction higher have positive SHAP. Features that pushed it lower have negative SHAP.

Top 5 features by mean absolute SHAP:

1. memory_utilization_pct 61,072 ← dominant
2. graphics_encoded 57,540 ← surprising finding
3. thermal_memory_interaction 24,489
4. swap_utilization_pct 23,989
5. swapfree_kb 16,252

Finding 1 — Memory Utilization Dominates

memory_utilization_pct is the strongest predictor with a mean SHAP of 61,072 KB. This is the normalized ratio of used memory to total RAM — not raw bytes but percentage.

The insight here is that the normalized ratio beats raw byte values completely. swapfree_kb (raw bytes) only ranks fifth with SHAP of 16,252. The model learned that how full memory is relative to the device's total RAM matters more than absolute bytes. A device with 5GB used out of 6GB (83%) is in a fundamentally different pressure state than a device with 5GB used out of 12GB (42%), even though both have the same raw bytes used.

Research finding:
> "Memory utilization as a normalized percentage of device RAM is the dominant predictor of reclaim magnitude, outperforming all raw byte metrics — establishing that relative memory pressure rather than absolute consumption drives kernel reclaim decisions."

Finding 2 — Graphics Setting is Second Most Important

graphics_encoded ranks second with a mean SHAP of 57,540 KB. This is nearly as influential as memory utilization itself. This is the most surprising finding in the project.

Graphics quality outranks game identity (game_pressure_encoded is only ninth), device RAM, CPU load, and even raw swap state. A game at ultra graphics creates more memory pressure than a different game at low graphics regardless of which game it is. The rendering workload quality setting is a stronger signal of memory stress than the game title.

Research finding:
> "Graphics quality setting is the second strongest predictor of memory reclaim magnitude, outranking game identity, device RAM, and CPU load — suggesting that rendering workload intensity is a more fundamental driver of memory pressure than application-specific behavior."

Finding 3 — Thermal and Memory Interact

thermal_memory_interaction — the product of AP temperature and memory utilization percentage — ranks third with SHAP of 24,489 KB. This is an engineered feature that captures the interaction between two subsystems. Neither temperature alone nor memory alone tells the full story. A hot device under memory pressure reclaims more aggressively than a cool device under the same memory pressure.

This finding makes physical sense. A hot processor runs less efficiently, thermal throttling may be reducing CPU performance, and the system attempts to compensate by freeing more memory to reduce overall workload. The interaction term captures this joint effect that neither raw signal captures individually.

Research finding:
> "The interaction between thermal state and memory pressure is the third strongest predictor, ranking above either signal alone — confirming that multi-signal stress modeling captures reclaim behavior that single-metric approaches miss entirely."

Why These Three Findings Are Valuable

Each finding has a direct engineering implication:

Finding 1 (utilization % beats raw bytes):
LMKD should use normalized utilization thresholds per device RAM class, not fixed byte thresholds. A 500MB threshold makes sense on 6GB but not on 12GB.

Finding 2 (graphics > game identity):
Game Booster should adjust memory reservation based on current graphics settings, not just which game is running. Same game at ultra needs more aggressive pre-freeing than at low.

Finding 3 (thermal × memory interaction):
Thermal management and memory management should be coordinated, not independent. When AP temp crosses 65°C and memory is above 75%, the system should pre-free more aggressively than either threshold would trigger independently.

—----------

One Hard Question

The hardest question you will get is:

> "Graphics setting is your second most important feature. But the user controls graphics settings. How does a system-level component know what graphics setting is currently active?"

The answer is:

> "The graphics quality setting is accessible at runtime through the Android GameManager API introduced in Android 12. The GPU rendering quality tier is also readable from the game's configuration files in the app data directory. Additionally, DMA buffer size and GPU utilization indirectly encode the active graphics quality even without reading the setting directly — a model trained on explicit graphics labels could be deployed using DMA buffer and GPU utilization as proxies in production environments where direct settings access is restricted."

That answer shows you thought about deployment, not just model training.

—----------

How Will You Deploy Memory Reclaim Prediction — Explanation

The Core Idea

This is an on-device prediction system. Deployment means packaging the trained XGBoost model as a lightweight background service that runs during active gaming sessions, reads memory state every few seconds, predicts how much memory will be reclaimed, and signals the memory manager to pre-free the right amount before LMKD fires.No internet connection needed. No cloud server. Everything runs locally on the device itself.

Step by Step Deployment

Step 1 — Convert and Compress the Model

During development we save the model as a pickle file. That is fine for a laptop but too heavy for production on a mobile device.

For deployment we export XGBoost as a lightweight JSON model file. XGBoost natively supports this format and it loads significantly faster than pickle. We also apply quantization — reducing the precision of model weights from float32 to int8. This shrinks model size by roughly 4x with almost no accuracy loss.

Development format : xgb_regressor.pkl (~15MB)
Production format : xgb_model.json   (~3MB after quantization)

A 3MB model file is completely acceptable on any modern mobile device with 6GB to 12GB RAM.

Step 2 — Package All Artifacts Together

The model alone is not enough for deployment. Four artifacts must travel together:

```
xgb_model.json      → the trained XGBoost model weights
scaler.pkl          → StandardScaler fitted on training data
iqr_bounds.json     → IQR bounds computed from training sessions
game_profiles.json  → game pressure and category encodings
```

The scaler and IQR bounds are critical. If you deploy the model without them, or worse if you refit them on the new device's data, the features going into the model will be on a completely different scale than what the model was trained on. Predictions will be completely wrong. These artifacts must always travel with the model.

Step 3 — Build the Background Service

The inference engine runs as an Android background service that wakes up at the start of every gaming session and takes memory snapshots during gameplay.

Here is what the service does every 30 seconds during an active gaming session:

```
Wake up
      ↓
Detect active game
  read foreground app package name
  look up game profile (pressure level, category)
      ↓
Read memory state from kernel files
  /proc/meminfo      → memtotal, swapfree
  DMA buffer stats   → dma_buf_kb
  Heap metrics       → heapsize, heapmaxfree, heapminfree
  DHA metrics        → dha_empty, dha_cache
  /proc/stat         → currentcpuload
  Thermal sensor     → aptemp
      ↓
```

```
      ↓
Read game context
  graphics setting   → from GameManager API
  resolution         → from Display API
  refresh rate       → from Display API
  online/offline     → from ConnectivityManager
      ↓
Apply IQR bounds     → from iqr_bounds.json
Apply scaler         → from scaler.pkl
Compute engineered features
  memory_utilization_pct
  swap_utilization_pct
  thermal_memory_interaction
  etc.
      ↓
Run XGBoost inference
  predicted_reclaim_kb = model.predict(features)
      ↓
Signal memory manager
  pre_free(predicted_reclaim_kb)
      ↓
Log prediction and actual outcome
      ↓
Sleep 30 seconds
```

Step 4 — Pre-freeing Action Based on Prediction

The prediction drives a specific system action. The memory manager uses the predicted reclaim amount to decide how aggressively to pre-free memory before LMKD fires:

```
Predicted reclaim < 100MB:
  Light pre-freeing
  Trim cached processes slightly
  User notices nothing

Predicted reclaim 100MB - 300MB:
  Moderate pre-freeing
  Release cached background apps
  Compress inactive process memory
```

Predicted reclaim 300MB - 600MB:
  Aggressive pre-freeing
  Kill low-priority cached apps
  Release file cache pages

Predicted reclaim > 600MB:
  Maximum pre-freeing
  Kill all non-essential background processes
  Release all reclaimable cached memory
  Notify Game Booster to reduce graphics quality

The key benefit here is that pre-freeing happens gradually and proactively rather than all at once reactively. LMKD killing 600MB in one sudden event causes game stutter. Pre-freeing 600MB gradually over 30 seconds before LMKD fires is invisible to the user.

Step 5 — Handle the Game Context Correctly

One deployment challenge specific to this project is reading graphics settings at runtime. During training we captured this manually. In production there are three ways to read it:

Option 1 — GameManager API (Android 12+)
  GameManager.getGameMode() returns
  GAME_MODE_STANDARD, GAME_MODE_PERFORMANCE,
  or GAME_MODE_BATTERY
  Map these to low/medium/high/ultra equivalents

Option 2 — GPU utilization as proxy
  If GameManager API unavailable
  read GPU busy percentage from
  /sys/class/kgsl/kgsl-3d0/gpu_busy_percentage
  High GPU busy% at lower resolution = high graphics
  This is an indirect but reliable proxy

Option 3 — Game configuration files
  Some games write their graphics setting to
  shared preferences accessible via adb
  Read directly if available

Step 6 — Log Every Prediction

Every prediction gets logged locally on the device:

```
timestamp
game_name
graphics_setting
predicted_reclaim_kb
actual_reclaim_kb     ← recorded after reclaim happens
error_kb              ← difference between predicted and actual
action_taken          ← which pre-freeing level was triggered
```

This log serves two purposes. First it lets you monitor whether the model is still performing well over time. If MAE starts drifting above 150MB consistently, the model needs retraining. Second every logged session becomes new training data for the next model version. The model improves with every device it runs on.

Step 7 — Update Models Over the Air

When a new model version is ready — because a new game is released, a new device series launches, or Android OS update changes memory behavior — the new model files get pushed via OTA (Over The Air) update through the standard firmware update mechanism. The background service loads the new files on its next activation. The user does not need to do anything.

```
Training team retrains model with new data
        ↓
New artifacts pushed via OTA
  xgb_model.json (new version)
  scaler.pkl     (new version)
  iqr_bounds.json (new version)
        ↓
Background service detects new files on next boot
        ↓
Loads new model automatically
        ↓
Old model files deleted
```

Inference Speed

XGBoost with 500 trees on 34 features : ~1ms
Feature engineering from snapshot : ~2ms
File reads from kernel : ~5ms
Total per 30-second cycle : ~8ms

CPU overhead : less than 0.03% of total CPU time
Memory usage : less than 5MB total
Battery impact: negligible

Running an 8ms computation every 30 seconds on a modern mobile processor is essentially free. This is why tree-based models are strongly preferred over neural networks for on-device deployment — they are extremely fast at inference time.

"The entire inference pipeline — reading kernel files, computing features, applying scaler and IQR bounds, running XGBoost, and signaling the memory manager — completes in under 10 milliseconds every 30 seconds, meaning the monitoring overhead is less than 0.03% of CPU time, which is essentially invisible from a device resource perspective."
